



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrian Davila Perez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 3

Integrante(s): 323001315, 426057857, 323008754, 426058665,
323067267

No. de lista o

brigada: 5

Semestre: 2027-1

Fecha de entrega: 11 de septiembre del 2026

Índice

1. Introducción	2
1.1. Planteamiento del problema	2
1.2. Motivación	2
1.3. Objetivos	2
2. Marco Teórico	2
2.1. Bibliotecas de Java	2
2.2. API de Java	3
2.3. Arreglos	3
2.4. Manejo de cadenas	3
2.5. Clase <code>StringBuilder</code>	3
2.6. Clases de utilería	3
3. Desarrollo	4
4. Resultados	6
4.1. Palindromo.	6
4.2. <code>HashMap</code> y <code>TreeSet</code>	7
5. Conclusiones	8

1. Introducción

1.1. Planteamiento del problema

Durante el desarrollo de programas es común encontrar tareas que requieren funcionalidades que ya han sido implementadas previamente. En Java existen bibliotecas que contienen clases y métodos que permiten realizar operaciones comunes sin necesidad de desarrollar nuevamente dichas funcionalidades. La Práctica 3, denominada *Utilerías y clases de uso general*, plantea el uso de estas bibliotecas y algunas de sus clases para resolver diferentes necesidades dentro de un programa.

1.2. Motivación

Resolver estos ejercicios permite fortalecer el pensamiento lógico-computacional y el manejo eficiente de estructuras de control, habilidades indispensables para comprender más adelante los principios de la POO, como la encapsulación y la reutilización de código.

1.3. Objetivos

- Conocer las bibliotecas disponibles en el lenguaje Java.
- Utilizar clases propias de las bibliotecas del lenguaje.
- Aplicar utilerías de Java para resolver tareas comunes dentro de un programa.
- Reforzar el uso de clases y métodos proporcionados por las bibliotecas estándar.
- Conocer las bibliotecas disponibles en el lenguaje Java.
- Utilizar clases propias de las bibliotecas del lenguaje.
- Aplicar utilerías de Java para resolver tareas comunes dentro de un programa.
- Reforzar el uso de clases y métodos proporcionados por las bibliotecas estándar.

2. Marco Teórico

2.1. Bibliotecas de Java

Las bibliotecas de Java son conjuntos de clases que proporcionan funcionalidades que pueden ser utilizadas por los programas. Estas bibliotecas permiten aprovechar código previamente desarrollado para realizar tareas comunes, evitando la necesidad de implementar nuevamente funciones que ya existen [1].

El uso de clases preconstruidas permite reducir el tiempo de desarrollo y puede contribuir a mejorar la calidad del software, debido a que dichas clases han sido probadas y depuradas previamente [1].

2.2. API de Java

La API de Java proporciona la documentación de las clases que forman parte de las bibliotecas del lenguaje. A través de ella es posible consultar las clases disponibles, así como los métodos y funcionalidades que pueden utilizarse dentro de un programa [2].

El conocimiento de la API permite seleccionar las clases apropiadas para resolver determinadas necesidades del programa sin tener que desarrollar desde cero funcionalidades existentes.

2.3. Arreglos

Un arreglo es un objeto que permite almacenar un conjunto de datos del mismo tipo. Cada elemento posee un índice numérico que identifica su posición dentro del arreglo. En Java, el primer elemento corresponde al índice 0 [1].

Los arreglos pueden utilizarse para almacenar y procesar conjuntos de información de manera organizada. Para utilizarlos es necesario establecer el tipo de datos que almacenarán y su tamaño correspondiente.

2.4. Manejo de cadenas

En Java, las cadenas de caracteres son objetos pertenecientes a la clase **String**. Esta clase proporciona diferentes métodos para realizar operaciones relacionadas con el tratamiento de texto, como obtener la longitud de una cadena, comparar cadenas, obtener caracteres y modificar su representación [3].

Entre los métodos proporcionados por la clase **String** se encuentran `length()`, `equals()`, `charAt()`, `substring()`, `toUpperCase()` y `toLowerCase()`, entre otros [3].

2.5. Clase **StringBuilder**

La clase **StringBuilder** permite trabajar con secuencias de caracteres cuyo contenido puede modificarse durante la ejecución del programa. A diferencia de los objetos de tipo **String**, cuyo contenido es inmutable, un objeto **StringBuilder** puede modificar su contenido [3].

Entre sus operaciones principales se encuentran `append()`, que permite agregar caracteres al final de la secuencia, e `insert()`, que permite agregar caracteres en una posición determinada.

2.6. Clases de utilería

Java proporciona clases de utilidad que permiten realizar operaciones generales, como cálculos matemáticos y manejo de fechas. Estas clases proporcionan métodos que pueden ser utilizados directamente para resolver tareas específicas dentro de una aplicación [1].

Una de estas clases es `Math`, que proporciona métodos para realizar operaciones matemáticas habituales, como calcular potencias mediante `pow()` y raíces cuadradas mediante `sqrt()` [1].

3. Desarrollo

Ejercicio. Funcionamiento de `HashMap` y `TreeSet` con 3 ejemplos comunes

1. **Inicio de la investigación en equipo.** Cada integrante del equipo realizará la investigación sobre `HashMap` y `TreeSet` de forma independiente, consultando cada uno con su propio LLM.
2. **Consulta del Integrante 1.** Se le pregunta a la IA sobre el funcionamiento interno de `HashMap` y `TreeSet`, solicitando también un ejemplo de uso de cada estructura.
3. **Consulta del Integrante 2.** Se le plantea al LLM la misma investigación, buscando conocer cómo funcionan ambas estructuras y qué comportamiento muestran ante distintos casos de uso.
4. **Consulta del Integrante 3.** Se investiga con este LLM el funcionamiento de `HashMap` y `TreeSet`, indagando en el mecanismo interno que utiliza cada una para organizar sus datos.
5. **Consulta del Integrante 4.** Se le pide al LLM que explique ambas estructuras, con énfasis en posibles aplicaciones o escenarios donde convendría usar cada una.
6. **Consulta del Integrante 5.** Se investiga con este LLM el comportamiento de `HashMap` y `TreeSet`, incluyendo casos particulares o restricciones que pudieran no ser evidentes a primera vista.
7. **Comparación de resultados entre integrantes.** Una vez reunidas las cinco investigaciones, se contrastan entre sí para observar en qué coinciden y en qué difieren las respuestas obtenidas por cada LLM.
8. **Identificación de hallazgos relevantes.** Se revisan los resultados en conjunto para detectar algún dato, comportamiento o restricción que alguno de los integrantes haya encontrado y que valga la pena resaltar.

Ejercicio 2. Palíndromo

1. **Inicio del programa.** Se crea un objeto de la clase `Scanner` para permitir la lectura de datos desde el teclado.
2. **Declaración de variables.** Se declara la variable número de tipo `long`, que almacenará el valor ingresado por el usuario.

3. **Control del ciclo.** Se establece un ciclo do-while que solicita el número al usuario y se repite mientras número sea menor o igual a 9999, o mayor o igual a 100000.
4. **Captura de datos.** Dentro del ciclo, se solicita al usuario ingresar un número, el cual se almacena en la variable número.
5. **Validación del rango.** Se verifica si número no tiene 5 dígitos (es decir, si es menor o igual a 9999 o mayor o igual a 100000).
 - Si la condición se cumple, se muestra un mensaje de error indicando que el número debe tener 5 dígitos.
 - El ciclo vuelve a evaluarse y, si sigue sin cumplirse el rango, se repite la solicitud del número.
6. **Fin del ciclo.** Una vez que número está dentro del rango de 5 dígitos (entre 10000 y 99999), el ciclo termina.
7. **Conversión de tipo.** Como ya se garantizó que número tiene 5 dígitos, se convierte de long a int y se almacena en la variable num.
8. **Extracción y comparación de dígitos.** Se invoca el método estático esPalindromo(num), el cual:
 - Obtiene el primer dígito con $\text{num} / 10000$ y el último con $\text{num} \% 10$.
 - Obtiene el segundo dígito con $(\text{num} / 1000) \% 10$ y el cuarto con $(\text{num} / 10) \% 10$.
 - Compara ambos pares de dígitos: si los dos pares coinciden, el método regresa true; de lo contrario, regresa false.
9. **Evaluación del resultado.** Con base en el valor booleano que regresa esPalindromo, se imprime en pantalla `.Es un palindromo.` o `"No es un palindromo"`.
10. **Cierre del programa.** Se cierra el objeto Scanner y finaliza la ejecución.

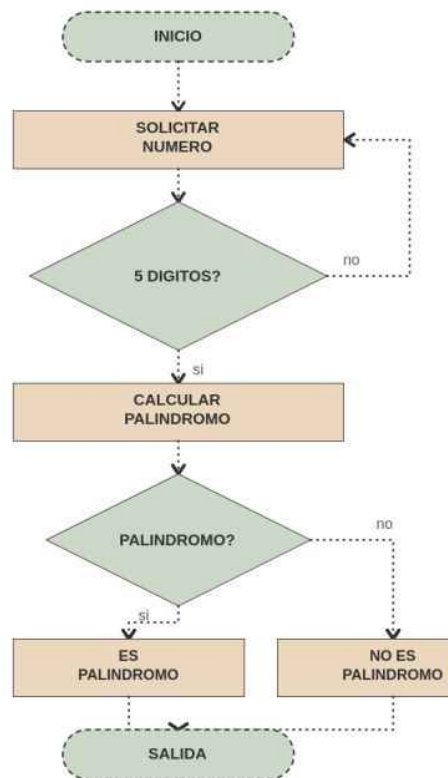


Figura 1: Diagrama de flujo ejercicio 2

4. Resultados

4.1. Palindromo.

El primer programa creado fue un verificador de palíndromos; solicitaba al usuario que introdujera cinco números y determinaba si la secuencia era un palíndromo o no, o bien, si se excedía el límite de números pedía al usuario que volviera a introducir cinco números.. .

```

PS C:\Users\acer\Desktop\P00> java Palindromo
Ingresa un numero de 5 digitos: 12321
Es un palindromo
PS C:\Users\acer\Desktop\P00> java Palindromo
Ingresa un numero de 5 digitos: 12345
No es un palindromo
PS C:\Users\acer\Desktop\P00> java Palindromo
Ingresa un numero de 5 digitos: 384959383
Error: el numero debe tener 5 digitos.
Ingresa un numero de 5 digitos: 45654
Es un palindromo
PS C:\Users\acer\Desktop\P00>

```

Figura 2: Palindromo

4.2. HashMap y TreeSet.

Al preguntar al LLM sobre el HashMap y TreeSet; nos dio lo siguiente:

HashMap (Velocidad extrema)

Guarda datos en pares **clave-valor**. Usa una fórmula matemática (*hash*) para ubicar y extraer la información casi instantáneamente ($O(1)$), pero **no ordena** los elementos.

3 Aplicaciones:

1. **Caché:** Guardar resultados pesados para acceso ultrarrápido (ej. datos de un perfil).
2. **Contadores:** Contar frecuencias (ej. cuántas veces aparece una palabra en un texto).
3. **Sesiones web:** Vincular instantáneamente un *token* de seguridad con los permisos de un usuario.

TreeSet (Orden automático)

Guarda **elementos únicos** en una estructura de árbol que los **ordena automáticamente** (alfabética o numéricamente) al insertarlos ($O(\log n)$).

3 Aplicaciones:

1. **Rankings:** Mantener tablas de puntuación ordenadas de mayor a menor (Top 10).

2. **Agendas:** Organizar citas o eventos de forma estrictamente cronológica y sin empalmes.
3. **Filtros de etiquetas:** Mostrar listas de *tags* únicas y ordenadas alfabéticamente en un blog.

Característica	HashMap	TreeSet
¿Qué guarda?	Pares de Clave-Valor (Ej. ID $\rightarrow$ Nombre).	Elementos individuales (Ej. Nombre).
Orden de los datos	Ninguno. Los datos se guardan desordenados.	Ordenado. Automáticamente de menor a mayor.
Estructura interna	Tabla Hash.	Árbol de búsqueda (Rojo-Negro).
Velocidad	Extrema ($O(1)$). Acceso casi instantáneo.	Rápida ($O(\log n)$). Tarda un poco más al reordenar.
¿Permite duplicados?	Las claves deben ser únicas, los valores pueden repetirse.	No. Ignora automáticamente los repetidos.
¿Permite nulos?	Sí (una clave <i>null</i> y varios valores <i>null</i>).	No (lanza error).
Uso ideal	Búsquedas ultrarrápidas usando un identificador.	Colecciones sin repetidos que deben estar siempre ordenadas.

Figura 3: `HashMap` `TreeSet`

5. Conclusiones

La aplicación de las utilerías y clases de uso general permitió comprender la importancia de aprovechar las funcionalidades que proporciona Java para resolver problemas de manera estructurada y eficiente. El uso de bibliotecas, arreglos y clases para el manejo de cadenas y operaciones específicas facilita la implementación de soluciones al evitar desarrollar desde cero funcionalidades que ya se encuentran disponibles. De esta manera, los conceptos estudiados permiten construir programas más organizados y aprovechar de mejor manera los recursos proporcionados por el lenguaje.

Referencias

- [1] A. Martín, *Programador Certificado Java 2*, 2.a ed. México: Alfaomega Grupo Editor, 2008.
- [2] K. Sierra y B. Bates, *SCJP Sun Certified Programmer for Java 6 Study Guide*. McGraw-Hill, 2008.

- [3] J. Dean y R. Dean, *Introducción a la programación con Java*, 1.a ed. México: McGraw-Hill, 2009.